DGL∞ — The landscape of generative GNNs

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.


https://github.com/udlbook/

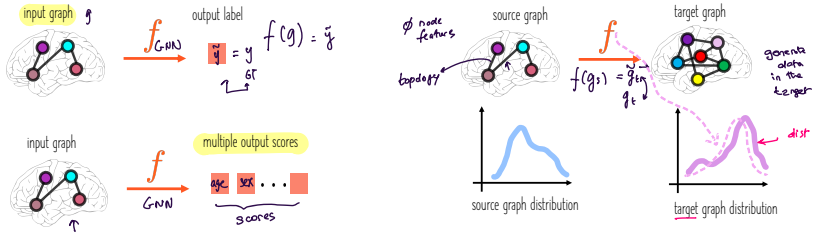
The landscape of generative GNNs

Generative learning tasks formalization

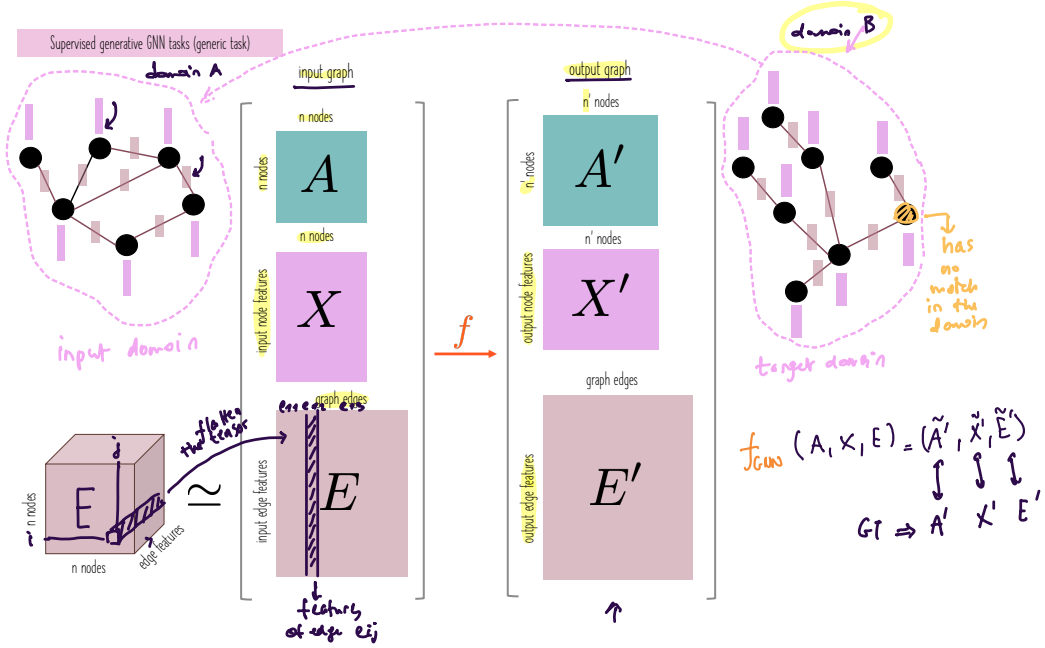
GNN aims

1) Predict and analyze patterns on given graphs (e.g., predict the label of a node or a graph)

2) Learn the distribution of given graphs and generate more novel graphs (e.g., generate target graph B from source graph A)



Supervised generative GNN tasks (generic task)

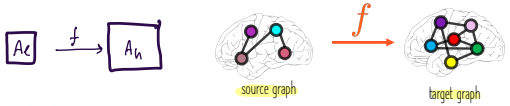


How to define the loss function?

This depends on the target data to generate (only A, A and X, etc), over which we will optimize our loss.

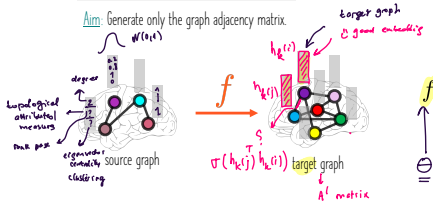
! Remember we need both A and X = H₀ matrices in the GCN node embedding rule.

How to solve the following generative task? We have no input X matrix.



The target graph has no node/edge features

Aim: Generate only the graph adjacency matrix.



GNN mapping function

$$f(A, X) = (\tilde{A}', \tilde{X}')$$

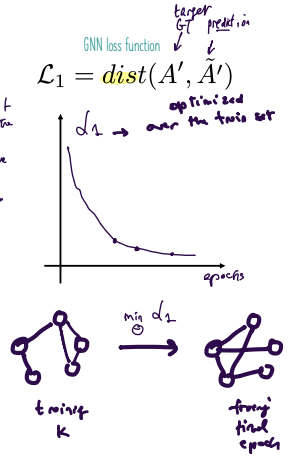
(GCNs)

Θ = all GNN params

We don't need the loss to include this → ∅ no GT X'

GNN loss function

$$\mathcal{L}_1 = dist(A', \tilde{A}')$$

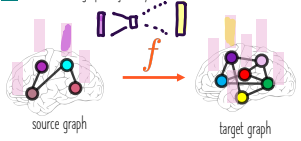


How to initialize $X = H_0$?

1. Randomly.
2. Using the topological attributes of a given node. (topology-driven)
3. Using an identity vector of [1 1 ... 1].

The target graph has only node features

Aim: Generate the graph adjacency matrix and its node feature matrix in the target domain.



GNN mapping function

$$f(A, X) = (\tilde{A}', \tilde{X}')$$

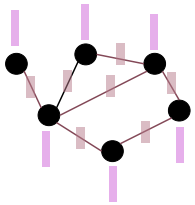
GNN loss function

$$\mathcal{L}_2 = dist(A', \tilde{A}') + \lambda\, dist(X', \tilde{X}')$$

L_2 (L_2 (sparsity))

The target graph has both node and edge features

Aim: Generate the graph adjacency matrix and its node and edge feature matrices in the target domain.



GNN mapping function

$$f(A, X, E) = (\tilde{A}', \tilde{X}', \tilde{E}')$$

balance terms over

GNN loss function

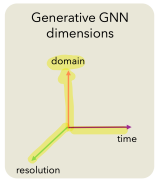
$$\mathcal{L}_3 = dist(A', \tilde{A}') + \lambda_1\, dist(X', \tilde{X}') + \lambda_2\, dist(E', \tilde{E}')$$

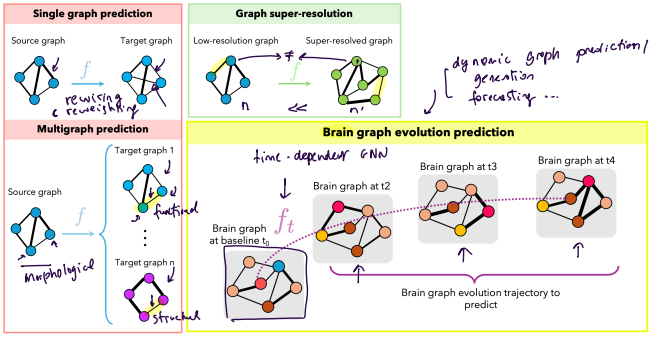
O(100) O(100) O(1)

topology — node features — edge features

so ↓ less attention

Various supervised (conditional) generative GNN tasks (Bessadok et al., 2022)





DGL∞ — The Landscape of Generative GNNs

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.

https://github.com/udlbook/

The landscape of generative GNNs

Generative learning tasks formalization

Self-supervised/unsupervised generative GNN tasks ⇒ no longer corresponding GT graph given g_s generate g_t. (clustering)

Graph generation: Previously, we learned about encoding graph structure by generating node embeddings based on local network neighbourhood.
Graph neural networks are different from traditional neural networks in a sense that we want the neural network to capture feature information and also the graph structure.
Graph encoders → graph to embeddings → encoded / embedded a given graph g_E eg h_G → g (decoding)
Graph decoders → embeddings to graphs = generative power, given an embedding h_G → g (decoding)

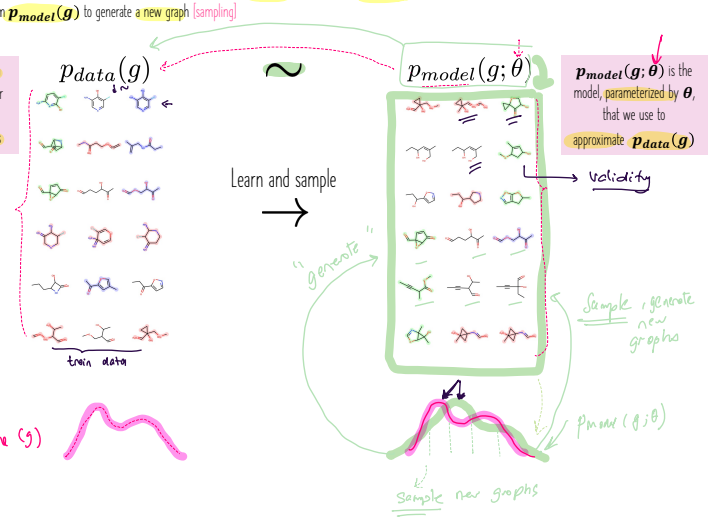
Utility: Graph generation is useful to understand graph formation process, identifying abnormal parts of the graph, predicting new structures, simulations of novel graph structures, in some cases graph completion in case of partially available graph data has real. (a few nodes / edges are available)
Example 1: generate a new kind of molecule that cures a certain disease, given a class of molecules like toxic/non-toxic we want new molecules that satisfy this class.
Example 2: generate the dynamic evolution brain connectivity graph to spot atypical alterations that may be linked to neurological disorders.

ML basics for self-supervised/unsupervised graph generation

Given graphs sampled from $p_{data}(g)$, we aim to:
1) learn a good model that represents a graph, in other words learn the distribution $p_{model}(g;\theta) \sim p_{data}(g)$ (density estimation)
2) identify how sample from $p_{model}(g)$ to generate a new graph (sampling)

$p_{data}(g)$ is the data distribution, which is never known to us, other than existing samples of graphs

$p_{model}(g;\theta)$ is the model, parameterized by θ, that we use to approximate $p_{data}(g)$



1- (density estimation)
Use the maximum likelihood to make $p_{model}(g;\theta) \sim p_{data}(g)$

$$\theta^* = \arg\max_\theta \mathbb{E}_{g\sim p_{data}} log p_{model}(g;\theta)$$

Find the parameters θ^* such that for the observed graph points $g_i \sim d_{data}$ the $\sum_i log\, p_{model}(g_i;\theta^*)$ has the highest value, among all possible choices of θ.
→ find the model that is mostly likely to have generated the observed graphs g.

self-regeneration
↳ self-supervised (unsupervised)

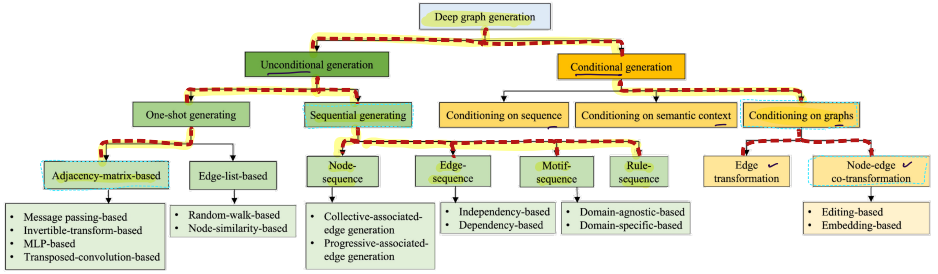
2- (sampling)
Sample from the complex distribution $p_{model}(g;\theta^*)$
The most common approach samples from a simple noise distribution:

$$z_i \sim \mathcal{N}(0,1)$$

Then transforms the noise z_i via a deep network function $f(.)$ trained on the input graphs:

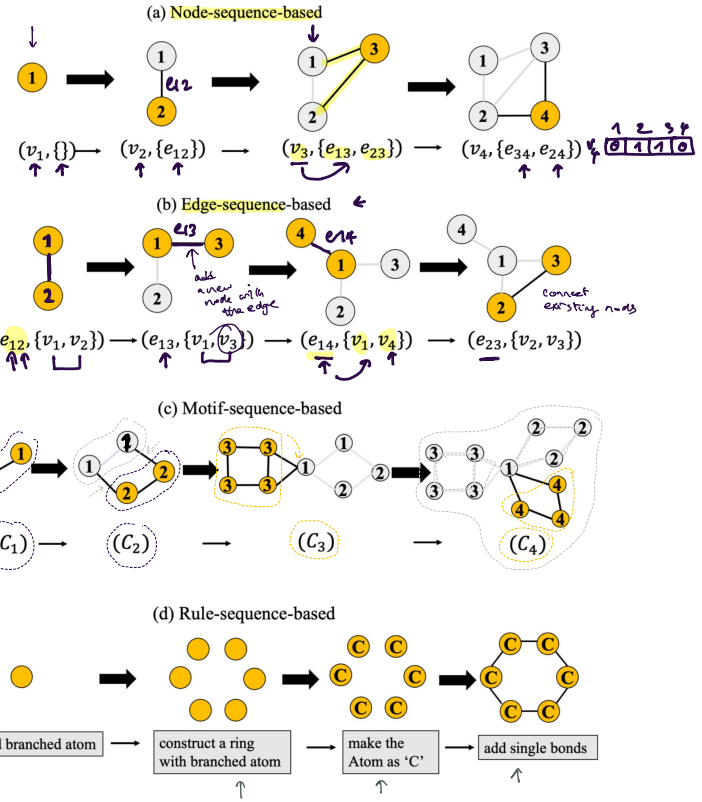
$$g_i = f(z_i;\theta^*)$$

Taxonomy (Guo et al., 2023)



Unconditional generation → does not always imply that we are in a fully unsupervised framework

Sequential generating



DGL∞ — The Landscape of Generative GNNs

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.


https://github.com/udlbook/

The landscape of generative GNNs

Taxonomy (Guo et al., 2023)

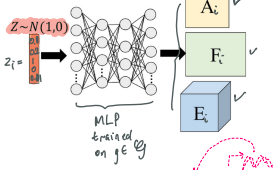
Self-supervised/unsupervised generative GNN tasks (self-generation)

Unconditional generation

One-shot generating

Adjacency-matrix-based

(a) MLP-based one-shot



GraphVAE (Simonovsky et al., 2018)

GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders

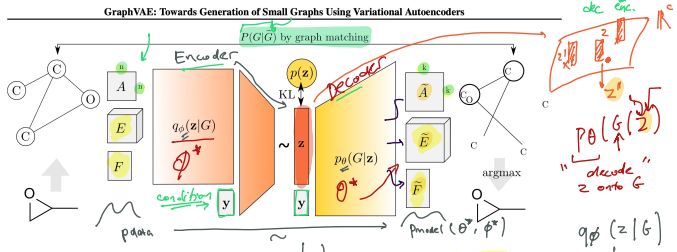


Figure 1. Illustration of the proposed variational graph autoencoder. Starting from a discrete attributed graph $G = (A, E, F)$ on n nodes (*e.g.* a representation of propylene oxide), stochastic graph encoder $q_\phi(\mathbf{z}|G)$ embeds the graph into continuous representation $\mathbf{z}$. Given a point in the latent space, our novel graph decoder $p_\theta(G|\mathbf{z})$ outputs a probabilistic fully-connected graph $\bar{G} = (\bar{A}, \bar{E}, \bar{F})$ on predefined $k \geq n$ nodes, from which discrete samples may be drawn. The process can be conditioned on label $\mathbf{y}$ for controlled sampling at test time. Reconstruction ability of the autoencoder is facilitated by approximate graph matching for aligning $\bar{G}$ with G.

Loss objective of variational autoencoders

Let $G = (A, E, F)$ be a graph specified with its adjacency matrix A, edge attribute tensor E, and node attribute matrix F. We wish to learn an encoder and a decoder to map between the space of graphs G and their continuous embedding $\mathbf{z} \in \mathbb{R}^d$ see Figure 1. In the probabilistic setting of a VAE, the encoder is defined by a variational posterior $q_\phi(\mathbf{z}|G)$ and the decoder by a generative distribution $p_\theta(G|\mathbf{z})$, where ϕ and θ are learned parameters. Furthermore, there is a prior distribution $p(\mathbf{z})$ imposed on the latent code representation as a regularization; we use a simplistic isotropic Gaussian prior $p(\mathbf{z}) = \mathcal{N}(0, I)$. The whole model is trained by minimizing the upper bound on negative log-likelihood $-\log p_\theta(G)$ (Kingma & Welling, 2013):

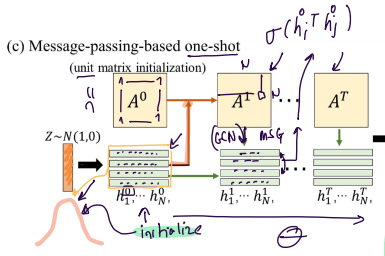
$$\mathcal{L}(\phi, \theta; G) = \mathbb{E}_{q_\phi(\mathbf{z}|G)}[-\log p_\theta(G|\mathbf{z})] + \mathrm{KL}[q_\phi(\mathbf{z}|G)||p(\mathbf{z})] \quad (1)$$

The first term of $\mathcal{L}$, the reconstruction loss, enforces high similarity of sampled generated graphs to the input graph G. The second term, KL-divergence, regularizes the code space to allow for sampling of $\mathbf{z}$ directly from $p(\mathbf{z})$ instead from $q_\phi(\mathbf{z}|G)$ later. The dimensionality of $\mathbf{z}$ is usually fairly small so that the autoencoder is encouraged to learn a high-level compression of the input instead of learning to simply copy any given input. While the regularization is independent on

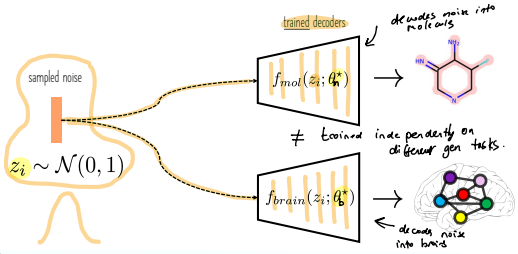
Decoder architecture

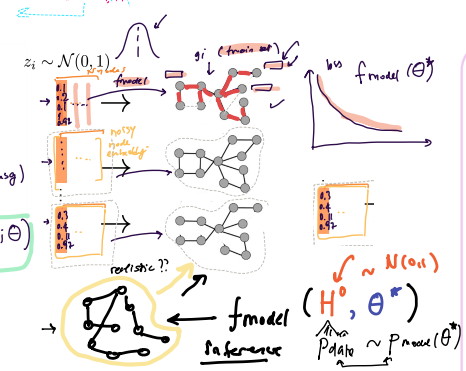
The decoder itself is deterministic. Its architecture is a simple multi-layer perceptron (MLP) with three outputs in its last layer. Sigmoid activation function is used to compute $\bar{A}$, whereas edge- and node-wise softmax is applied to obtain $\bar{E}$ and $\bar{F}$, respectively. At test time, we are often interested in a (discrete) point estimate of $\bar{G}$, which can be obtained by taking edge- and node-wise argmax in $\bar{A}$, $\bar{E}$, and $\bar{F}$. Note that this can result in a discrete graph on less than k nodes.

(c) Message-passing-based one-shot



The universal generative power: transforming noise to real graphs





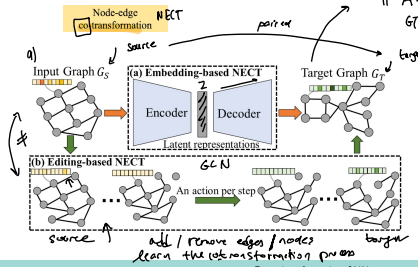
Taxonomy (Guo et al., 2023)

Supervised generative GNN tasks

Conditional generation

Conditioning on graphs

Node-edge co-transformation NECT



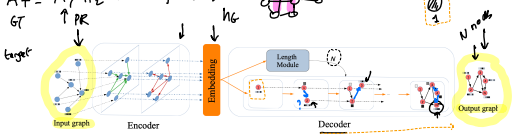
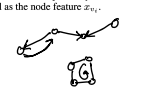


Figure 1: The encoder embeds of the input graph. The decoder generates an output graph conditioned on this embedding by first determining the number of nodes N, then generating N nodes sequentially. At each step n, edges from existing nodes to the new node v_i are determined as well as the node feature x_{v_i}.

DAG-to-DAG (Kaluza t al., 2018)

Directed-acyclic-graph (DAG)



Encoder-Decoder GNNs case study: Graph UNet (Gao et al., 2019)

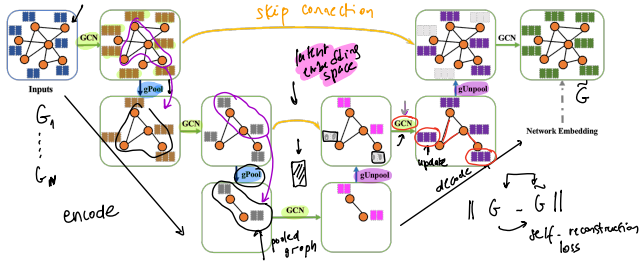


Figure 3. An illustration of the proposed graph U-Nets (g-U-Nets). In this example, each node in the input graph has two features. The input feature vectors are transformed into low-dimensional representations using a GCN layer. After that, we stack two encoder blocks, each of which contains a gPool layer and a GCN layer. In the decoder part, there are also two decoder blocks. Each block consists of a gUnpool layer and a GCN layer. For blocks in the same level, encoder block uses skip connection to fuse the low-level spatial features from the encoder block. The output feature vectors of nodes in the last layer are network embedding, which can be used for various tasks such as node classification and link prediction.

Proposed graph pooling – downsample

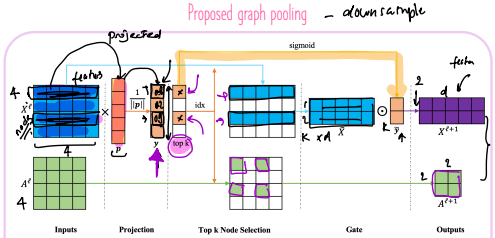


Figure 1. An illustration of the proposed graph pooling layer with $k = 2$. $\times$ and $\odot$ denote matrix multiplication and element-wise product, respectively. We consider a graph with 4 nodes, and each node has 5 features. By processing this graph, we obtain the adjacency matrix $A^l \in \mathbb{R}^{4\times4}$ and the input feature matrix $X^l \in \mathbb{R}^{4\times5}$ of layer l. In the projection stage, $\mathbf{p} \in \mathbb{R}^5$ is a trainable projection vector. By matrix multiplication and sigmoid$(\cdot)$, we obtain $\mathbf{y}$ that are scores estimating scalar projection values of each node to the projection vector. By using $k = 2$, we select two nodes with the highest scores and record their indices in the top-k-node selection stage. We use the indices to extract the corresponding nodes to form a new graph, resulting in the pooled feature map $\tilde{X}^l$ and new corresponding adjacency matrix A^{l+1}. At the gate stage, we perform element-wise multiplication between $\tilde{X}^l$ and the selected node scores vector $\tilde{\mathbf{y}}$, resulting in X^{l+1}. This graph pooling layer outputs A^{l+1} and X^{l+1}.

higher values

$$\mathbf{y} = X^l\mathbf{p}^l/\|\mathbf{p}^l\|, \quad \in \mathbb{R}^n$$
$$\mathrm{idx} = \mathrm{rank}(\mathbf{y}, k),$$
$$\tilde{\mathbf{y}} = \mathrm{sigmoid}(\mathbf{y}(\mathrm{idx})),$$
$$\tilde{X}^l = X^l(\mathrm{idx}, :),$$
$$A^{l+1} = A^l(\mathrm{idx}, \mathrm{idx}),$$
$$X^{l+1} = \tilde{X}^l \odot (\tilde{\mathbf{y}}\mathbf{1}_d^T),$$

new pooled graph.

Proposed graph unpooling – upsample

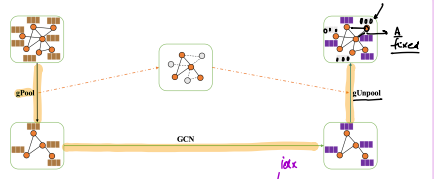


Figure 2. An illustration of the proposed graph unpooling (gUnpool) layer. In this example, a graph with 7 nodes is down-sampled using a gPool layer, resulting in a coarsened graph with 4 nodes and position information of selected nodes. The corresponding gUnpool layer uses the position information to reconstruct the original graph structure by using empty feature vectors for unselected nodes.

DGL∞ — The landscape of generative GNNs

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.

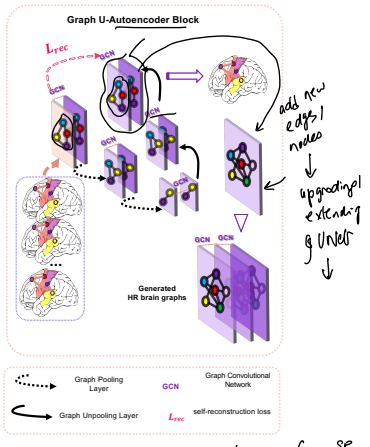

https://github.com/udlbook/

The landscape of generative GNNs

Encoder-Decoder GNNs case study: Graph UNet (Gao et al., 2019)

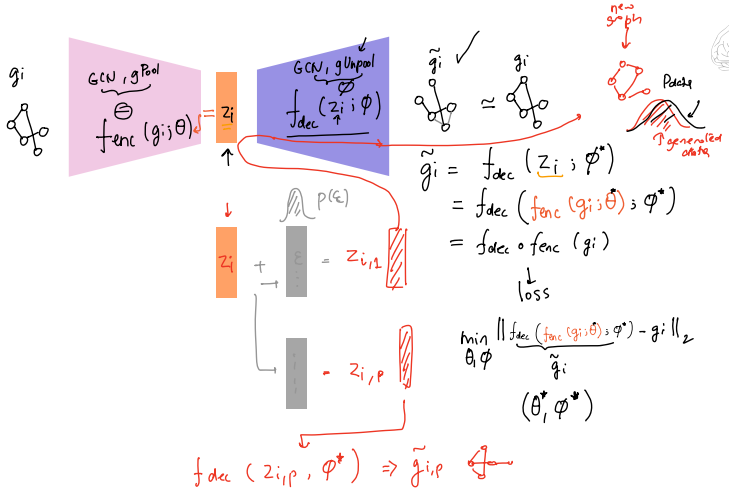
How can we use Graph UNet for super-resolving graphs?

GSR-Net (Isallari et al., 2018)

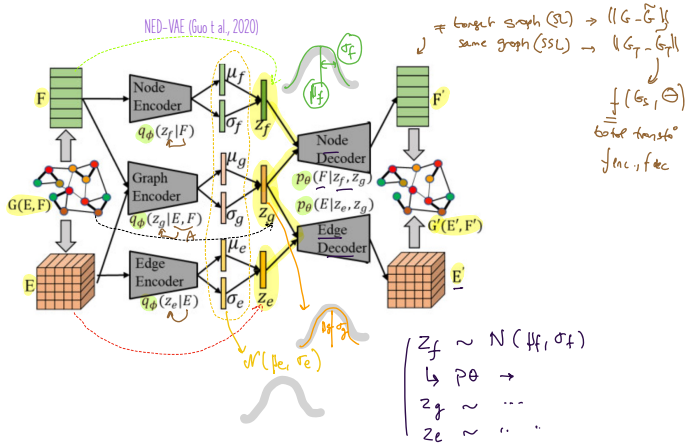


→ maybe add a new block for sr.

→ How to generate new graphs?

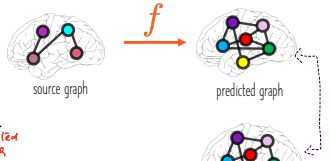


How to generalize graph UNet to a target with also edge attributes?

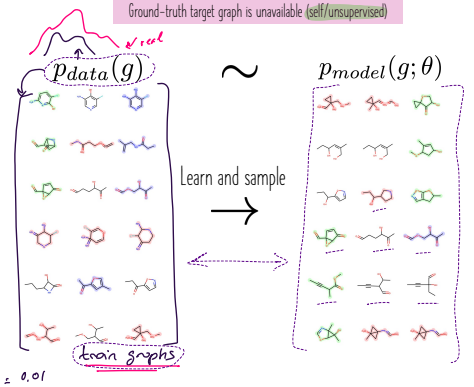


Evaluation measures for graph generation

1) Ground-truth target graph is available (supervised)



Ground-truth target graph is unavailable (self/unsupervised)



dist (g_t, $\tilde{g}_t$)

1) MAE (A_t, $\tilde{A}_t$) = 0.01 (KL-divergence)

2) topological means - soundness

MAE (eigenvector centrality)

degree

Extra reading resources

Published as a conference paper at ICLR 2022

On Evaluation Metrics for Graph Generative Models

Rylee Thompson[1,2], Boris Knyazev[1,2,3], Elahe Ghalebi[2], Jungtaek Kim[4], Graham W. Taylor[1,2]
[1] University of Guelph, [2] Vector Institute, [3] Samsung, SAIT AI Lab, Montreal, [4] POSTECH
{rylee, bknyazev, gwtaylor}@uoguelph.ca
elahe.ghalebi@vectorinstitute.ai, jtkim@postech.ac.kr

ABSTRACT

In image generation, generative models can be evaluated naturally by visually inspecting model outputs. However, this is not always the case for graph generative models (GGMs), making their evaluation challenging. Currently, the standard process for evaluating GGMs suffers from three critical limitations: i) it does not produce a single score which makes model selection challenging, ii) in many cases it fails to consider underlying edge and node features, and iii) it is prohibitively slow to perform. In this work, we mitigate these issues by searching for *scalar, domain-agnostic, and scalable metrics* for evaluating and ranking GGMs. To this end, we study existing GGM metrics and neural-network-based metrics emerging from generative models of images that use embeddings extracted from a task-specific network. Motivated by the power of Graph Neural Networks (GNNs) to extract meaningful graph representations *without any training*, we introduce several metrics based on the features extracted by an untrained random GNN. We design experiments to thoroughly test and objectively score metrics on their ability to measure the diversity and fidelity of generated graphs, as well as their sample and computational efficiency. Depending on the quantity of samples, we recommend one of two metrics from our collection of random-GNN-based metrics. We show these two metrics to be more expressive than pre-existing and alternative random-GNN-based metrics using our objective scoring. While we focus on applying these metrics to GGM evaluation, in practice this enables the ability to easily compute the dissimilarity between any two sets of graphs *regardless of domain.* Our code is released at: https://github.com/uoguelph-mlrg/GGM-metrics.

A Systematic Survey on Deep Generative Models for Graph Generation

Xiaojie Guo and Liang Zhao

Evaluation Metrics for Deep Generative-Based Methods

Type		*Evaluation feature*
General	Statistics-based	Average KLD, MMD
	Classifier-based	Accuracy-based, FID-based
	Intrinsic-quality-based	Validity, Uniqueness, Novelty
Condition-specialized	Graph property-based	
	Mapping-relationship-based	

KLD : KL divergence between distributions
MMD : Maximum Mean Discrepancy
FID : Fréchet Inception Distance between multivariate Gaussian distributions.

References:
1. Taxonomy: Guo, X., & Zhao, L. (2022). A systematic survey on deep generative models for graph generation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(5), 5370-5390.
2. GraphVAE: Simonovsky, M., & Komodakis, N. (2018). Graphvae: Towards generation of small graphs using variational autoencoders. In Artificial Neural Networks and Machine Learning-ICANN 2018: 27th International Conference on Artificial Neural Networks, Rhodes, Greece, October 4–7, 2018
3. DAG-to-DAG: Kaluza, M. C. D. P., Amizadeh, S., & Yu, R. (2018). A neural framework for learning DAG to DAG translation. In NeurIPS'2018 Workshop.
4. Graph U-Net: Gao, H., & Ji, S. (2019, May). Graph U-Nets. In international conference on machine learning (pp. 2083-2092). PMLR.
5. Evaluation: Thompson, R., Knyazev, B., Ghalebi, E., Kim, J., & Taylor, G. W. (2022). On evaluation metrics for graph generative models. arXiv preprint arXiv:2201.09871. MSc thesis.
6. NED-VAE: Guo, X., Zhao, L., Qin, Z., Wu, L., Shehu, A., & Ye, Y. (2020, August). Interpretable deep graph generation with node-edge co-disentanglement.
7. https://wandb.ai/syllogismos/machine-learning-with-graphs/reports/10-Deep-Generative-Models-for-Graphs--Vmlldzox0zcwMTI

BASIRA https://basira-lab.com/

YouTube Search "BASIRA Lab"

https://github.com/basiralab/DGL